PROVISIONAL PATENT APPLICATION

SYSTEM AND METHOD FOR CONTROLLING ACCESS TO STORED DATA THROUGH COMPOSABLE POLICY-ENFORCING INTERFACES WITH CONJUNCTIVE MULTI-MECHANISM ACCESS CONTROL AND FAILURE INDISTINGUISHABLE FROM NONEXISTENCE

Authors: Tom Tang

Organization: Edge Vector Foundation

Date: April 30, 2026

Applicant

Edge Vector Foundation
14205 SE 36th Street, Suite 100, Bellevue, WA 98006

Inventors

- Tom Tang, Seattle, WA

Field of the Invention

This invention relates to access control in database systems, and more particularly to a system and method in which stored data is not directly addressable by callers and is reachable only through named, composable interface definitions that independently enforce per-field access policies at each step of a directed acyclic graph of derived projections, returning, on any policy failure, an output indistinguishable from the response that would be returned if the queried interface did not exist.

Cross-Reference to Related Applications

None (provisional filing).

Note: This application claims an access-control architecture that is independent of, and distinct from, the inventions claimed in the co-pending provisional applications by the same applicant titled "System and Method for Automated Schema Deduplication and Canonical Field Resolution Using Semantic Similarity with Information-Theoretic Transform Classification" (which claims

schema reconciliation and canonical naming) and "System and Method for Validating Safety and Correctness Properties of User-Submitted Code Executing Inside a Database Through Sandboxed WebAssembly Modules with Content-Addressed Verification" (which claims sandboxed transform execution). The present application claims the architectural pattern by which data is made unreachable except through composable policy-enforcing interfaces; it does not claim any particular schema-management or transform-execution mechanism.

Background of the Invention

Conventional database access control operates at a perimeter. A caller is authenticated, granted privileges over tables, schemas, or columns, and thereafter issues queries that read or write the underlying data. Access enforcement happens once, on the way in. Once data has been read out of the system, the system has no further influence over it.

Existing fine-grained mechanisms — SQL views, row-level security ("RLS"), column masking, and stored procedures — partially mitigate this by interposing logic between the caller and the underlying tables. However, each suffers from limitations that prevent it from constituting a sole, composable access path:

1. **Views are not the sole access path.** A view in a relational database is a convenience layer over base tables; the base tables remain directly addressable to any caller with the underlying privilege. A privileged caller, a misconfigured grant, or a direct database connection bypasses the view entirely.

2. **Row-level security does not compose with independent policy at each step.** RLS attaches predicates to rows of a table; predicates do not compose across derived views with each derived view independently re-evaluating its own policy. A chain of derived views inherits the base table's predicate without per-step enforcement.

3. **Stored procedures and user-defined functions execute logic but do not constrain the result projection.** A stored procedure can return any subset of any data the procedure itself can read; the caller's policy does not gate the projection independently from the procedure's authorship.

4. **Failure modes leak structure.** Conventional access control distinguishes "permission denied" from "no such object" via distinct error codes. This permits a caller to enumerate the existence of objects to which it has no read access, and to distinguish authorization decisions from data absence.

5. **Multiple access patterns over the same datum require duplicated storage.** A practitioner who wants to expose the same underlying value to two different audiences with two different policies — for example, a clinical view and a de-identified research view of the same patient record — typically must materialize two copies and keep them synchronized.

6. **Economic gates are external.** Pay-per-query access, when present, is implemented at an API gateway external to the access-control system. The data layer cannot itself condition a read on payment status, and pricing cannot be parameterized by the caller's relationship to the data owner.

There is a need for an access-control architecture in which (i) stored data is not directly addressable and is reachable only through named interfaces, (ii) those interfaces compose into derived projections with each composition step independently enforcing its own policy, (iii) failure at any step yields an output that does not disclose which step failed or whether the queried interface even exists, and (iv) the policy at each step can integrate trust, capability, information-flow, and economic checks under a uniform conjunctive evaluation.

Summary of the Invention

The present invention provides a method, system, and computer-readable medium for controlling access to stored data through composable policy-enforcing interfaces, comprising:

1. **Interfaces as the sole access path.** Stored field values are not directly addressable by any caller. The only mechanism by which a caller can read or write a stored value is to evaluate an interface that exposes that value (directly or transitively).

2. **Composition into a directed acyclic graph (DAG).** An interface may derive one or more of its exposed fields from fields exposed by another interface via a transform. The collection of all interfaces in the system thereby forms a DAG, with cycles rejected at interface-registration time to guarantee termination.

3. **Per-step independent policy enforcement.** When a query is directed to a target interface, the system traverses the DAG from the target interface toward its sources and, at each interface visited, independently evaluates that interface's per-field access policy against the caller's access context. A successful query requires every visited interface's policy to be satisfied.

4. **Failure indistinguishable from nonexistence.** When any visited interface's policy is not satisfied for any of its exposed fields, the query returns an output to the caller that is indistinguishable from the response the system would return if the target interface did not exist. The caller cannot distinguish denial of access from absence of the queried interface, cannot determine which interface or which field caused the denial, and cannot enumerate the existence of interfaces to which it has no read access.

5. **Conjunctive multi-mechanism policy.** Each interface's per-field access policy is the conjunction of (i) a trust-distance check, (ii) a cryptographic-capability check, (iii) an information-flow label check over a security lattice, and (iv) a payment check. Failure of any mechanism for any field of any visited interface yields the indistinguishable-from-nonexistence output.

6. **Additive trust distance with shortest-path resolution and explicit owner override.** Trust between users is represented by a directed graph of explicit non-negative integer assignments. The effective trust distance between a caller and a data owner is the minimum, over all directed paths in the graph, of the sum of explicit assignments along each path, except where the data owner has declared an explicit assignment for that caller, in which case the explicit value takes precedence over any derived value.

7. **Bounded-quota cryptographic capabilities.** A field may carry capability records, each binding a public key to an integer quota counter that decrements per access and revokes

access on reaching zero. Quota counters cannot be incremented in place; the data owner increases available quota only by issuing a new capability record and revoking the prior one, with both actions recorded in an append-only audit log.

8. **Payment as a monotone non-decreasing function of trust distance.** Each interface may associate with itself a price function $C : \mathbb{N}_0 \to \mathbb{R}_{\geq 0}$ that is monotonically non-decreasing in trust distance. A query against the interface is accepted only when the caller has paid at least $C(\tau)$, where τ is the caller's effective trust distance to the data owner. Payment is evaluated independently per interface; payment for one interface confers no payment status on any other.

9. **Multiple interfaces over the same stored datum.** Two or more interfaces in the plurality may each expose the same stored field, with different policies, different transforms, or both, such that the same stored value is simultaneously reachable through multiple interfaces yielding different projections under different policies, without duplicating the underlying storage.

10. **Audit decoupled from data evaluation.** Every query, whether successful or yielding the indistinguishable-from-nonexistence output, is recorded in an append-only audit log identifying the caller, the target interface, and the outcome. The audit record is written even when the caller receives no data and no indication that any policy decision was made.

11. **Write-through derived interfaces via declared inverses.** A transform may declare an inverse function. A write directed to a field of a derived interface whose transform has a declared inverse is propagated to the corresponding source interface by applying the inverse, with the propagated write independently subjected to the source interface's per-field write policy. The append-only store retains all prior versions of every source field.

Detailed Description of the Invention

Architecture Overview

A system embodying the invention comprises:

- A **data store** holding stored field values, structurally inaccessible to any caller except through interface evaluation.
- A **interface registry** maintaining the plurality of interface definitions, their per-field access policies, their transforms, and the dependency graph among interfaces.
- An **execution engine** that, in response to a query directed to a target interface under an access context, traverses the dependency graph and evaluates per-interface policies.
- An **audit log** that records every query and every administrative change to interface definitions, trust assignments, and capability records.

The novel architectural property is that the data store exposes no interface other than interface evaluation. There is no "table read" primitive accessible to callers, no "raw value" addressing by

primary key, no introspection that would reveal field values outside an interface projection. The interface is structurally — not by convention — the only entry point.

Interfaces

An interface definition specifies:

- An interface identifier.
- A set of exposed fields, each having a name, a type, and a per-field access policy.
- For each exposed field, optionally, a transform that derives the field's value from fields exposed by another (named) interface.
- The data owner whose policies govern the interface.

An interface whose exposed fields all derive from another interface is termed a *derived interface*; an interface whose exposed fields directly reference stored values is termed a *base interface*. Both kinds participate equally in the access-control architecture: a base interface is not privileged with respect to the data store, and its per-field policy is enforced on every access in the same manner as a derived interface's.

The Interface DAG

The plurality of interfaces collectively forms a directed acyclic graph in which an edge from interface F_k to interface F_{k-1} indicates that F_k has at least one exposed field whose transform sources from F_{k-1}. Cycles are rejected at interface-registration time; an interface definition whose introduction would create a cycle is refused.

Query Evaluation and Per-Step Policy

When a caller issues a query directed to a target interface under an access context (identifying the caller, the caller's claimed cryptographic keys, and the caller's payment status with respect to the target interface), the execution engine performs the following:

1. Traverses the DAG from the target interface toward its sources.
2. At each interface visited in the traversal, independently evaluates that interface's per-field access policy against the access context for every field whose value is required to compute the target interface's projection.
3. If every required per-field policy at every visited interface is satisfied, materializes the target interface's projection by applying transforms in topological order from sources to target, and returns the projection to the caller.

4. If any required per-field policy at any visited interface is not satisfied, returns to the caller an output indistinguishable from the response the system would return if the target interface did not exist.

The independence of per-step evaluation is essential. A caller authorized at the target interface but unauthorized at an intermediate interface does not receive the target interface's projection; conversely, a caller authorized at all intermediate interfaces but unauthorized at the target receives no data. The composition is conjunctive across the entire traversal.

The Indistinguishable-from-Nonexistence Output

On any policy failure, the system returns to the caller an output that does not disclose:

- whether the target interface exists,
- whether the caller is authorized for any other interface,
- which interface or which field caused the failure,
- which of the four conjunctive mechanisms (trust, capability, label, payment) caused the failure, or
- whether the failure occurred at the target interface or at any ancestor in the DAG.

This is enforced at the protocol layer: error codes, response sizes, response timing, and response structure are normalized across "interface does not exist," "interface exists but caller has no read permission," "interface exists, caller has read permission, but an ancestor interface's policy denied," and analogous cases. The audit log records the actual reason for the failure; the caller does not.

Conjunctive Per-Field Policy

Each per-field access policy is the conjunction of four checks.

Trust-Distance Check. A directed graph of explicit non-negative integer *trust assignments* is maintained, in which an edge from user a to user b labeled n records that a has declared b's trust distance from a to be n. The effective trust distance from caller u to data owner o is computed as the minimum, over all directed paths from o to u in this graph, of the sum of edge labels along the path. If o has declared an explicit assignment $\tau(u, o)$, that explicit value overrides the computed minimum.

Each field exposes thresholds $W_n R_m$. A write to the field is permitted only if the effective trust distance is at most n; a read is permitted only if the effective trust distance is at most m.

Trust assignments are mutable. Changes to trust assignments take effect on the next query; in-flight queries complete under their original effective distance, and no results already returned are retroactively withdrawn. There is no caching of effective trust distances across queries; each query resolves the current minimum.

Cryptographic-Capability Check. A field may carry zero or more *capability records*, each specifying a public key and an integer quota counter for read access, write access, or both. On each successful access by a holder of the corresponding private key, the relevant counter is decremented by one. When a counter reaches zero, the capability is revoked and access via that capability is refused.

Quota counters cannot be modified in place. To increase the quota available to a particular public key, the data owner issues a new capability record and revokes the prior one. Both the issuance and the revocation are recorded in the append-only audit log.

When both a trust-distance threshold and a capability constraint are present on a field, the caller must satisfy both. Neither overrides the other.

Information-Flow Label Check. Each field carries a *security label* ℓ drawn from a lattice $(\mathcal{L}, \sqsubseteq)$. A transform that produces a derived field f_j from a source field f_i is permitted only if $\ell_i \sqsubseteq \ell_j$: information flows to equal or higher security levels and never downward. The check is enforced both statically at transform registration and dynamically at query time, since label assignments may change after registration.

Payment Check. An interface may associate with itself a price function $C : \mathbb{N}_0 \to \mathbb{R}_{\geq 0}$ that is monotonically non-decreasing in trust distance — closer users pay no more than distant ones. The interface owner is unrestricted in the choice of function beyond this monotonicity constraint; common choices include constant, linear, and exponential. A query against the interface is accepted only when the caller has paid at least $C(\tau)$ for that interface, where τ is the caller's effective trust distance to the data owner.

The payment predicate is evaluated independently per interface. Payment for one interface confers no payment status on any other interface; the same caller may have paid for some interfaces in a chain and not for others, and the payment check at each interface in the traversal is decided on its own.

Multiple Interfaces over the Same Stored Datum

The architecture permits — and is intended to be used with — two or more interfaces each exposing the same stored field, with differing per-field access policies, differing transforms, or both. For example, a clinical interface may expose a patient's name unchanged with a tight trust-distance threshold, while a research interface exposes the same stored name through an irreversible hashing transform with a looser threshold. The same underlying value is reachable through both interfaces; each enforces its own policy independently; no duplication of the underlying storage is required.

Audit Independence

An append-only audit log records every query, every administrative change to interface definitions, every trust-assignment change, every capability issuance and revocation, and every payment event. The audit record for a query is written regardless of whether the query returned a projection or

the indistinguishable-from-nonexistence output, and identifies the actual cause of any failure. The audit log is structurally distinct from the data store: data flows through interfaces; audit records flow into the audit log without policy gating.

Write-Through Derived Interfaces

A transform may optionally declare an inverse function. When a write is directed to a field of a derived interface whose transform has a declared inverse, the system applies the inverse to the proposed value and propagates the resulting value as a write to the corresponding field of the source interface. The propagated write is independently subjected to the source interface's per-field write policy. The append-only store retains all prior versions of every source field, so concurrent writes to the same source via different derived interfaces may set different last-write-wins active values without erasing history; prior versions remain available for audit and rollback.

A transform without a declared inverse is read-only when accessed through a derived interface. Writes directed to its exposed fields are refused with the same indistinguishable-from-nonexistence semantics applied to read failures.

Security Properties

The architecture provides the following structural guarantees, expressible as properties of the system rather than requirements on caller behavior:

- **Sole-access invariant.** No stored field value can be read or written by any caller except by evaluating an interface that exposes the field directly or transitively.
- **Per-step independence.** A caller authorized at the target interface but unauthorized at any intermediate interface receives no data.
- **Failure indistinguishability.** A caller cannot distinguish denial of access from absence of the queried interface, nor determine which mechanism produced the denial.
- **Audit completeness.** Every access decision, whether resulting in disclosure or denial, is recorded.
- **No retroactive disclosure.** Changes to trust assignments or capability records do not retroactively withdraw results already returned, and no caching of policy decisions persists across queries.

Claims

1. A computer-implemented method for controlling access to stored data, comprising:

   (a) maintaining, on a node, a set of stored field values;

(b) maintaining a plurality of named interface definitions" each interface definition specifying (i) a set of fields exposed by the interface, (ii) for each exposed field, a per-field access policy, and (iii) optionally, a transform that derives the value of the exposed field from one or more fields exposed by another interface, such that the plurality of interfaces collectively forms a directed acyclic graph;

(c) restricting access to the stored field values such that the field values are not directly addressable by a caller and are reachable only by evaluating an interface;

(d) in response to a query directed to a target interface under an access context identifying a caller, evaluating the target interface by traversing the directed acyclic graph from the target interface toward its source interfaces and, at each interface visited in the traversal, independently evaluating that interface's per-field access policy against the access context;

(e) when the per-field access policy of every visited interface is satisfied, returning to the caller a projection of the fields exposed by the target interface; and

(f) when the per-field access policy of any visited interface is not satisfied for any exposed field whose value is required, returning to the caller an output that is indistinguishable from the response the system would return if the target interface did not exist, such that the caller cannot distinguish denial of access from nonexistence and cannot determine which interface, which field, or which check produced the denial.

2. The method of claim 1, wherein the per-field access policy is a conjunction of (i) a trust-distance check, (ii) a cryptographic-capability check, (iii) an information-flow label check over a lattice, and (iv) a payment check, and wherein failure of any one of the four checks against any field of any visited interface causes the query to return the output indistinguishable from nonexistence.

3. The method of claim 2, wherein the trust-distance check comprises:

(a) maintaining a directed graph of explicit non-negative integer trust assignments declared by users;

(b) computing, for the caller and a data owner, an effective trust distance equal to the minimum, over all directed paths from the data owner to the caller in the graph, of the sum of the explicit trust assignments along the path;

(c) overriding the computed effective distance with an explicit assignment declared by the data owner for the caller, when present, such that explicit owner assignments take precedence over derived values; and

(d) evaluating the trust-distance check for a field having policy thresholds $W_n R_m$ by permitting a write only if the effective trust distance is at most n, and permitting a read only if the effective trust distance is at most m.

4. The method of claim 2, wherein the cryptographic-capability check comprises:

(a) associating with a field zero or more capability records, each capability record specifying a public key and an integer quota counter for read access, write access, or both;

(b) on each successful access by a holder of the corresponding private key, decrementing the quota counter by one;

(c) rejecting access when the quota counter reaches zero; and

(d) prohibiting in-place modification of the quota counter, such that the data owner increases available quota only by issuing a new capability record and revoking the prior one, both actions recorded in an append-only audit log.

5. The method of claim 2, wherein the payment check comprises:

   (a) associating with each interface a price function $C : \mathbb{N}_0 \to \mathbb{R}_{\geq 0}$ that is monotonically non-decreasing in trust distance;

   (b) accepting a query against the interface only when the caller has paid at least $C(\tau)$ for that interface, where τ is the caller's effective trust distance to the data owner; and

   (c) evaluating the payment check independently per interface, such that prior payment for one interface confers no payment status on any other interface.

6. The method of claim 1, wherein two or more interfaces in the plurality each expose the same stored field, but with different per-field access policies, different transforms, or both, such that the same stored value is simultaneously reachable through multiple interfaces yielding different projections under different policies, without duplicating the storage of the underlying field value.

7. The method of claim 1, further comprising writing to an append-only audit log, on each query, a record identifying the caller, the target interface, and the outcome, regardless of whether the query returned a projection or the output indistinguishable from nonexistence, and wherein the audit record identifies the actual cause of any policy failure even though the caller does not.

8. The method of claim 1, further comprising rejecting at interface-registration time any interface definition whose introduction would create a cycle in the directed acyclic graph of interface dependencies, thereby guaranteeing termination of query evaluation.

9. The method of claim 1, wherein a transform optionally declares an inverse, and wherein a write directed to a field of a derived interface whose transform has a declared inverse is propagated to the corresponding source interface by applying the declared inverse, with the propagated write independently subjected to the source interface's per-field write policy and recorded as a new entry in an append-only store that retains all prior versions of the source field.

10. The method of claim 1, wherein modifications to trust assignments, capability records, or payment status take effect on subsequent queries and do not retroactively withdraw projections already returned to callers under prior access contexts.

11. A system comprising:

    (a) one or more processors;

    (b) a data store holding stored field values;

    (c) an interface registry holding a plurality of interface definitions as recited in claim 1; and

    (d) memory storing instructions that, when executed by the one or more processors, cause the system to perform the method of any of claims 1–10.

12. A non-transitory computer-readable medium storing instructions that, when executed by one or more processors, cause the processors to perform the method of any of claims 1–10.

Drafting Notes (Internal — Not for Filing)

- **Distinguishing over SQL views.** Views project data but do not constitute the *sole* access path; base tables remain directly addressable. Claim 1(c) — "not directly addressable… reachable only by evaluating an interface" — is the structural distinction. The specification should emphasize that this is enforced architecturally, not by convention or grant configuration.

- **Distinguishing over row-level security.** RLS predicates do not compose across derived views with each step independently re-evaluating its own policy. Claim 1(d) — "at each interface visited… independently evaluating" — is the differentiator.

- **Distinguishing over capability systems (Dennis & Van Horn, KeyKOS, seL4).** Prior capability systems do not address composable derived projections with per-step policy at each composition node. Claim 4 layers bounded-quota counters with strict reissue-only semantics on top of the capability primitive.

- **Distinguishing over ABAC/RBAC.** Trust distance as a single additive scalar with shortest-path semantics and explicit owner override is distinct from attribute predicates or role hierarchies. Claim 3 captures this.

- **Distinguishing over pay-per-query data marketplaces.** Existing marketplaces gate access at an external API layer; the data layer itself does not condition reads on payment, and pricing is not parameterized by the caller's relationship to the owner. Claim 5 — price as a monotone function of trust distance, evaluated per-interface — is the differentiator.

- **The "indistinguishable from nonexistence" language in claim 1(f)** is doing significant work. Counsel may want to consider further broadening: response timing normalization, response size normalization, and prohibition of side-channel disclosures (e.g., differential audit-log behavior visible to the caller) could be added as a dependent claim or worked into the spec.

- **Claim 6 (multi-interface over same datum)** could be promoted to a second independent claim if counsel wants two parallel independents to maximize coverage. It captures the inventor's original framing — "schemas floating over the internal data" — most directly.

- **Cross-references** to `patent_schema_canonical_service` (schema reconciliation) and `patent_verified_fun` (transform execution) should be made explicit in the cross-reference section to disclaim overlap. The present application claims the *access architecture*; those claim *schema management* and *transform execution*. The interface concept appears in all three but is claimed at different levels: here as the access path, in `schema_canonical_service` as a named registered interface, and in `verified_function_execution` as the invocation context for a registered transform module.

- **Open question for counsel:** whether to file as a single provisional or split into two provisionals, one for the access architecture (claims 1, 6, 7, 8, 9, 10) and one for the conjunctive multi-mechanism policy (claims 2, 3, 4, 5). A single filing is simpler; splitting may produce stronger independent claims if the multi-mechanism policy turns out to be patentable independently of the interface architecture.

Prepared as a draft for provisional patent filing by Edge Vector Foundation.
Filing date: [TO BE DETERMINED]
This document is a working draft and has not been reviewed by patent counsel.